

Project Report: Hand Sign Recognition Using Computer Vision

Course: Computer Vision

Project Title: Real-Time Hand Sign Recognition Using MediaPipe and OpenCV

Date: September 2026

1. Introduction

The idea behind this project was pretty straightforward — I wanted to build something that could look at a hand through a webcam and figure out what gesture is being shown. Things like a thumbs up, peace sign, fist, pointing, etc. Hand gesture recognition has a lot of real-world uses, from controlling devices without touching them, to helping with sign language interpretation.

Instead of going the deep learning route (which would need a GPU and a bunch of training data), I went with Google's MediaPipe library. It already has a trained model that can detect hands and give you 21 landmark points on each hand. From there, I wrote my own logic to figure out which fingers are up or down, and mapped those combinations to specific gestures.

The whole thing runs in real-time on just a laptop CPU, which was one of my main goals — keeping it simple and accessible.

2. Problem Statement

Most hand gesture recognition systems out there either need special hardware like depth cameras (Kinect, Leap Motion) or require you to train a neural network from scratch with thousands of labeled images. For a course project, that's not really practical.

What I wanted was something that:

- Works with a regular webcam (no special hardware)
- Doesn't need a GPU
- Can run in real-time without lag
- Is easy to understand and modify

So the challenge was: can we get decent gesture recognition using just landmark positions and some math, without training any models ourselves?

3. Objectives

The main goals I set for this project were:

1. Get real-time hand detection working using MediaPipe
2. Extract the 21 landmark coordinates from detected hands

3. Figure out which fingers are extended and which are curled based on landmark positions
 4. Use that finger state data to classify at least 10 different gestures
 5. Show the results on screen with labels and bounding boxes
 6. Also support analyzing single images (not just webcam)
-

4. Literature Review

I looked into a few different approaches before deciding on MediaPipe:

Haar Cascades — This is the classic OpenCV approach. It's fast but only gives you a bounding box, not individual finger positions. Also, accuracy isn't great for hands since they change shape so much.

CNN-based detectors (YOLO, SSD) — These are accurate but you need a GPU for real-time performance and you'd need to collect/label training data.

Skin color segmentation — Some projects detect hands by looking for skin-colored regions. This breaks easily with different skin tones and lighting conditions.

MediaPipe Hands — This ended up being the best fit. Google trained it on around 30,000 real-world images. It uses two models internally: one to find the palm in the image (BlazePalm), and another to locate all 21 landmarks on the hand. The nice thing is it runs fast on CPU and works well across different skin tones.

For the gesture classification part, I considered training an SVM or using template matching, but honestly the rule-based approach (just checking which fingers are up) worked well enough for the gestures I wanted to detect. Plus it's way easier to understand and debug.

5. System Architecture

Here's basically how the system works from start to finish:

```
Webcam captures a frame
↓
MediaPipe detects hands and gives 21 landmarks per hand
↓
My code checks each finger: is the tip above or below the PIP joint?
↓
Based on the finger combination, it picks a gesture name
↓
OpenCV draws the landmarks, bounding box, and label on the frame
↓
Display on screen with FPS counter
```

The main components are:

- **OpenCV** handles the webcam input and all the drawing/display stuff
 - **MediaPipe** does the heavy lifting of finding hands and landmarks
 - **Custom Python logic** does the finger analysis and gesture classification
 - **HUD overlay** shows the gesture name, FPS, and hand info on screen
-

6. Methodology

6.1 How the Landmarks Work

MediaPipe gives you 21 points on each hand. The wrist is point 0, then each finger has 4 points (MCP, PIP, DIP, TIP) going from base to tip. So the thumb is points 1-4, index finger is 5-8, middle is 9-12, ring is 13-16, and pinky is 17-20.

Each point has x, y, and z coordinates. x and y are normalized between 0 and 1 (relative to image size), and z represents depth relative to the wrist.

6.2 Figuring Out if a Finger is Extended

This is the core logic. The idea is simple: if a finger is extended (pointing up), its TIP will be higher in the image than its PIP joint. In image coordinates, "higher" means a smaller y value. So:

```
if tip.y < pip.y → finger is extended  
if tip.y > pip.y → finger is curled
```

The thumb is a special case because it moves sideways, not up and down. So for the thumb I compare x coordinates instead. For a right hand, the thumb is extended if `tip.x < ip.x` (pointing left/outward).

6.3 Mapping Fingers to Gestures

Once I know which fingers are up, I just match against known patterns. For example:

- All fingers down = Fist
- All fingers up = Open Hand
- Only index and middle up = Peace sign
- Only thumb up (and pointing upward) = Thumbs up

Some gestures need extra checks. The OK sign, for instance — you can't just check if fingers are up or down. I had to measure the distance between the thumb tip and index tip. If they're really close together (less than 0.05 in normalized coordinates), and the other fingers are extended, it's an OK sign.

Here's the full mapping I used:

Gesture	Which fingers are up	Any extra checks?
Fist	None	—
Open Hand	All five	—
Peace	Index + middle	—
Thumbs Up	Only thumb	Thumb tip is above the MCP joint
Thumbs Down	Only thumb	Thumb tip is below the wrist
Pointing	Only index	—
OK Sign	Middle + ring + pinky	Thumb & index tips are touching
Rock On	Index + pinky	—

Call Me	Thumb + pinky	—
Spider-Man	Thumb + index + pinky	—
Three	Index + middle + ring	—
Four	Index + middle + ring + pinky	—

7. Implementation Details

7.1 Tools Used

- **Python 3.8+** — main language
- **OpenCV** — webcam capture, image processing, drawing
- **MediaPipe** — hand detection and landmarks
- **NumPy** — some array math

7.2 Project Files

```
computer vision/  
■■■■ hand_sign_recognition.py  # Everything is in this one file  
■■■■ requirements.txt          # pip dependencies  
■■■■ README.md                 # GitHub readme  
■■■■ PROJECT_REPORT.md        # This report
```

I kept everything in a single Python file to keep it simple. The main functions are:

- `get_finger_states()` — looks at the landmarks and returns which fingers are up/down
- `classify_gesture()` — takes finger states and returns a gesture name
- `process_frame()` — runs the full pipeline on one frame
- `draw_hud()` — draws the overlay with gesture name, FPS, etc.
- `detect_from_webcam()` — the main real-time loop
- `detect_in_image()` — processes a single image file

7.3 Settings I Tuned

MediaPipe has a few confidence thresholds you can adjust:

- **Detection confidence: 0.6** — I bumped this up from the default 0.5 to reduce false positives. Below 0.5 it would sometimes detect hands where there weren't any.
- **Tracking confidence: 0.5** — Left this at default. It controls how confident the tracker needs to be to keep following a hand between frames.
- **Max hands: 2** — Supports two hands at once. Could set this to 1 for slightly better performance.

For the OK sign, the distance threshold of 0.05 was found through trial and error. Too low and it barely ever triggers, too high and it fires when fingers are just close but not touching.

8. Results

8.1 Performance Numbers

The system runs at about **25-30 FPS** on my laptop (no dedicated GPU). Each frame takes under 50ms to process. With two hands in view, FPS drops slightly but stays above 20.

8.2 How Well Do the Gestures Work?

I tested each gesture about 20-30 times in different conditions. Here's roughly how they performed:

Gesture	How reliable?	Notes
Fist	Very good	Almost never fails
Open Hand	Very good	Very distinctive, easy to detect
Peace	Good	Sometimes confused with "pointing" if middle finger isn't fully extended
Thumbs Up	Good	Works best when hand is facing the camera straight on
Thumbs Down	Okay	Trickier because the hand orientation matters
Pointing	Very good	Hard to confuse with anything else
OK Sign	Decent	The distance threshold is a bit finicky
Rock On	Good	Pretty distinctive finger pattern
Three/Four	Okay	These can get confused with each other sometimes

8.3 What I Noticed

- **Lighting matters a lot.** In a well-lit room with even lighting, detection is solid. In dim light or with strong backlight, it struggles.
 - **MediaPipe handles different skin tones well.** This was a concern initially but it works consistently.
 - **The mirror flip makes it way more intuitive.** Without flipping the image, it feels backwards and confusing to use.
 - **Background doesn't matter much.** Even with a cluttered background, MediaPipe focuses on the hands reliably.
-

9. Challenges I Ran Into

OpenCV version issues — I initially installed OpenCV 5.0, which turns out removed CascadeClassifier (I was originally trying face detection). Had to downgrade to 4.x. Later switched the whole project to MediaPipe which works with any OpenCV version.

The thumb is weird — Unlike other fingers that go up and down, the thumb moves sideways. My initial logic treated all fingers the same and the thumb detection was completely wrong. Had to add special x-axis logic for it, and it also depends on whether it's a left or right hand.

Thumbs up vs thumbs down — Just knowing the thumb is extended isn't enough. I had to add extra checks comparing the thumb tip position to the MCP joint and wrist to determine if it's pointing up or down.

OK sign detection — This was the hardest gesture because it's not just about which fingers are up. I had to calculate the actual distance between thumb and index fingertips, and finding the right threshold took some experimentation.

Gestures that look similar — Three fingers up vs four fingers up is hard to distinguish if one finger is only partially extended. The rule-based approach doesn't handle "partially extended" fingers — it's either up or down.

Limitations

- Only works with static poses, not dynamic gestures like waving
 - Doesn't work well at extreme angles (hand pointing at/away from camera)
 - Rule-based approach means adding new gestures requires writing code, not just providing examples
 - Not suitable for full sign language recognition since many signs involve hand motion and orientation
-

10. Future Scope

Some things I'd like to add if I continue working on this:

- **Sign language support** — Train a proper classifier (maybe an SVM or small neural net) on the landmark data to recognize ASL letters
 - **Dynamic gestures** — Track hand movement over time to detect gestures like wave, swipe, or circle
 - **Gesture controls** — Map gestures to actual system actions like controlling volume or switching slides in a presentation
 - **Better classification** — Replace the rule-based approach with a machine learning model trained on the landmark coordinates
 - **Mobile version** — MediaPipe has mobile SDKs, so this could be ported to a phone app
-

11. Conclusion

The project ended up working better than I expected. Using MediaPipe for the hand detection part saved a ton of time — I didn't have to collect training data or train any models. The rule-based gesture classification is simple but gets the job done for the 12 gestures I defined.

The main takeaway is that you can build a pretty capable gesture recognition system without any deep learning on your end, as long as you have good landmark detection (which MediaPipe provides). The whole thing runs smoothly on a regular laptop at 25-30 FPS, which was one of the key requirements.

If I were to improve it further, the biggest upgrade would be replacing the rule-based classifier with a trained model — that would handle edge cases better and make it easier to add new gestures.

12. References

1. Zhang, F., et al. "MediaPipe Hands: On-device Real-time Hand Tracking." arXiv:2006.10214, 2020.
 2. Lugaresi, C., et al. "MediaPipe: A Framework for Building Perception Pipelines." arXiv:1906.08172, 2019.
 3. Google MediaPipe documentation — <https://mediapipe.dev/>
 4. OpenCV documentation — <https://docs.opencv.org/>
 5. Rautaray, S. S. and Agrawal, A. "Vision based hand gesture recognition for human computer interaction: a survey." Artificial Intelligence Review, 43(1), 1–54, 2015.
 6. Pisharady, P. K. and Saerbeck, M. "Recent methods and databases in vision-based hand gesture recognition: A review." Computer Vision and Image Understanding, 141, 152–165, 2015.
-

Submitted as part of Computer Vision coursework — September 2026